

# Apply

In this lecture we will learn about 3 different `apply()` functions. The basic idea of an `apply()` is to apply a function over some iterable object.

Let's start with `lapply()`:

## `lapply()`

`lapply()` will apply a function over a list or vector:

```
lapply(X, FUN, ...)
```

where `X` is your list/vector and `FUN` is your function. For more info you can use:

In [1]:

```
help(lapply)
```

Out[1]:

|               |                 |
|---------------|-----------------|
| lapply {base} | R Documentation |
|---------------|-----------------|

## Apply a Function over a List or Vector

### Description

`lapply` returns a list of the same length as `X`, each element of which is the result of applying `FUN` to the corresponding element of `X`.

`sapply` is a user-friendly version and wrapper of `lapply` by default returning a vector, matrix or, if `simplify = "array"`, an array if appropriate, by applying `simplify2array().sapply(x, f, simplify = FALSE, USE.NAMES = FALSE)` is the same as `lapply(x, f)`.

`vapply` is similar to `sapply`, but has a pre-specified type of return value, so it can be safer (and sometimes faster) to use.

`replicate` is a wrapper for the common use of `sapply` for repeated evaluation of an expression (which will usually involve random number generation).

`simplify2array()` is the utility called from `sapply()` when `simplify` is not false and is similarly called from `mapply()`.

### Usage

```
lapply(X, FUN, ...)
```

```
sapply(X, FUN, ..., simplify = TRUE, USE.NAMES = TRUE)
```

```
vapply(X, FUN, FUN.VALUE, ..., USE.NAMES = TRUE)
```

```
replicate(n, expr, simplify = "array")
```

```
simplify2array(x, higher = TRUE)
```

### Arguments

|                       |                                                                                                                                                                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>X</code>        | a vector (atomic or list) or an expression object. Other objects (including classed objects) will be coerced by <code>base::as.list</code> .                                                                                                                              |
| <code>FUN</code>      | the function to be applied to each element of <code>X</code> : see 'Details'. In the case of functions like <code>+</code> , <code>%*%</code> , the function name must be backquoted or quoted.                                                                           |
| <code>...</code>      | optional arguments to <code>FUN</code> .                                                                                                                                                                                                                                  |
| <code>simplify</code> | logical or character string; should the result be simplified to a vector, matrix or higher dimensional array if possible? For <code>sapply</code> it must be named and not abbreviated. The default value, <code>TRUE</code> , returns a vector or matrix if appropriate, |

|           |                                                                                                                                                                                                                                                                                                             |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| simplify  | whereas if <code>simplify = "array"</code> the result may be an array of “rank” ( <code>=length(dim(.))</code> ) one higher than the result of <code>FUN(X[[i]])</code> .                                                                                                                                   |
| USE.NAMES | logical; if <code>TRUE</code> and if <code>X</code> is character, use <code>X</code> as names for the result unless it had names already. Since this argument follows <code>...</code> its name cannot be abbreviated.                                                                                      |
| FUN.VALUE | a (generalized) vector; a template for the return value from <code>FUN</code> . See ‘Details’.                                                                                                                                                                                                              |
| n         | integer: the number of replications.                                                                                                                                                                                                                                                                        |
| expr      | the expression (a language object, usually a call) to evaluate repeatedly.                                                                                                                                                                                                                                  |
| x         | a list, typically returned from <code>lapply()</code> .                                                                                                                                                                                                                                                     |
| higher    | logical; if true, <code>simplify2array()</code> will produce a (“higher rank”) array when appropriate, whereas <code>higher = FALSE</code> would return a matrix (or vector) only. These two cases correspond to <code>sapply(*, simplify = "array")</code> or <code>simplify = TRUE</code> , respectively. |

## Details

`FUN` is found by a call to `match.fun` and typically is specified as a function or a symbol (e.g., a backquoted name) or a character string specifying a function to be searched for from the environment of the call to `lapply`.

Function `FUN` must be able to accept as input any of the elements of `X`. If the latter is an atomic vector, `FUN` will always be passed a length-one vector of the same type as `X`.

Arguments in `...` cannot have the same name as any of the other arguments, and care may be needed to avoid partial matching to `FUN`. In general-purpose code it is good practice to name the first two arguments `X` and `FUN` if `...` is passed through: this both avoids partial matching to `FUN` and ensures that a sensible error message is given if arguments named `X` or `FUN` are passed through `...`.

Simplification in `sapply` is only attempted if `X` has length greater than zero and if the return values from all elements of `X` are all of the same (positive) length. If the common length is one the result is a vector, and if greater than one is a matrix with a column corresponding to each element of `X`.

Simplification is always done in `vapply`. This function checks that all values of `FUN` are compatible with the `FUN.VALUE`, in that they must have the same length and type. (Types may be promoted to a higher type within the ordering logical < integer < double < complex, but not demoted.)

Users of S4 classes should pass a list to `lapply` and `vapply`: the internal coercion is done by the `as.list` in the base namespace and not one defined by a user (e.g., by setting S4 methods on the base function).

`lapply` and `vapply` are primitive functions.

## Value

For `lapply`, `sapply(simplify = FALSE)` and `replicate(simplify = FALSE)`, a list.

For `sapply(simplify = TRUE)` and `replicate(simplify = TRUE)`: if `X` has length zero or `n = 0`, an empty list. Otherwise an atomic vector or matrix or list of the same length as `X` (of length `n` for `replicate`). If simplification occurs, the output type is determined from the highest type of the return values in the hierarchy `NULL < raw < logical < integer < double < complex < character < list < expression`, after coercion of pairlists to lists.

`vapply` returns a vector or array of type matching the `FUN.VALUE`. If `length(FUN.VALUE) == 1` a vector of the same length as `X` is returned, otherwise an array. If `FUN.VALUE` is not an array, the result is a matrix with `length(FUN.VALUE)` rows and `length(X)` columns, otherwise an array `a` with `dim(a) == c(dim(FUN.VALUE), length(X))`.

The (Dim)names of the array value are taken from the `FUN.VALUE` if it is named, otherwise from the result of the first function call. Column names of the matrix or more generally the names of the last dimension of the array value or names of the vector value are set from `X` as in `sapply`.

## Note

`sapply(*, simplify = FALSE, USE.NAMES = FALSE)` is equivalent to `lapply(*)`.

For historical reasons, the calls created by `lapply` are unevaluated, and code has been written (e.g., `bquote`) that relies on this. This means that the recorded call is always of the form `FUN(X[[i]], ...)`, with `i` replaced by the current (integer or double) index. This is not normally a problem, but it can be if `FUN` uses `sys.call` or `match.call` or if it is a primitive function that makes use of the call. This means that it is often safer to call primitive functions with a wrapper, so that e.g. `lapply(11, function(x) is.numeric(x))` is required to ensure that method dispatch for `is.numeric` occurs correctly.

`is.numeric(x)` is required to ensure that method dispatch for `is.numeric` occurs correctly.

If `expr` is a function call, be aware of assumptions about where it is evaluated, and in particular what `...` might refer to. You can pass additional named arguments to a function call as additional named arguments to `replicate`: see ‘Examples’.

## References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

## See Also

`apply`, `tpapply`, `mapply` for applying a function to multiple arguments, and `rapply` for a recursive version of `lapply()`, `eapply` for applying a function to each entry in an environment.

## Examples

```
require(stats); require(graphics)

x <- list(a = 1:10, beta = exp(-3:3), logic = c(TRUE,FALSE,FALSE,TRUE))
# compute the list mean for each list element
lapply(x, mean)
# median and quartiles for each list element
lapply(x, quantile, probs = 1:3/4)
sapply(x, quantile)
i39 <- sapply(3:9, seq) # list of vectors
sapply(i39, fivenum)
vapply(i39, fivenum,
       c(Min. = 0, "1st Qu." = 0, Median = 0, "3rd Qu." = 0, Max. = 0))

## sapply(*, "array") -- artificial example
(v <- structure(10*(5:8), names = LETTERS[1:4]))
f2 <- function(x, y) outer(rep(x, length.out = 3), y)
(a2 <- sapply(v, f2, y = 2*(1:5), simplify = "array"))
a.2 <- vapply(v, f2, outer(1:3, 1:5), y = 2*(1:5))
stopifnot(dim(a2) == c(3,5,4), all.equal(a2, a.2),
          identical(dimnames(a2), list(NULL,NULL,LETTERS[1:4])))

hist(replicate(100, mean(rexp(10))))

## use of replicate() with parameters:
foo <- function(x = 1, y = 2) c(x, y)
# does not work: bar <- function(n, ...) replicate(n, foo(...))
bar <- function(n, x) replicate(n, foo(x = x))
bar(5, x = 3)
```

---

[Package *base* version 3.2.2]

Let's see how we can use this in its most practical use case, apply a custom function to a vector. First I want to show you a quick function (we will go over more utilities like this one later) which will allow us to pick a random sample from a vector:

In [33]:

```
# sample just 1 random number between 1 and 10
sample(x = 1:10,1)
```

Out[33]:

3

In [34]:

```
# vector
```

```

# vector
v <- c(1,2,3,4,5)

# our custom function
addrand <- function(x) {
  # Get a random number
  ran <- sample(x=1:10,1)

  # return x plus the random number
  return(x+ran)
}

# lapply()
lapply(v,addrand)

```

Out[34]:

```

1. 7
2. 11
3. 9
4. 5
5. 6

```

## Anonymous Functions

So you noticed that in the last example we had to write out an entire function to apply to the vector, but in reality that function is just doing something pretty simple, adding a random number. Do we really want to have to formally define an entire function for this? We don't want to, especially if we only plan to use this function a single time!

To address this issue, we can create an anonymous function (called this because we won't ever name it). Here's the syntax for an anonymous function in R:

```
function(a){code here}
```

This is a similar idea to lambda expressions in Python. So for example we can rewrite the previous function as an anonymous function and use `lapply()` with it:

In [35]:

```
v
```

Out[35]:

```
1 2 3 4 5
```

In [37]:

```

# Anon func with lapply()
lapply(v, function(a) {a+sample(x=1:10,1)})

```

Out[37]:

```

1. 2
2. 11
3. 8
4. 5
5. 10

```

Notice how its kind of implied that everything inside of the curly brackets `{}` will be returned. Here's a simpler example:

In [38]:

```
# adds two to every element
```

```
lapply(v,function(x){x+2})
```

Out[38]:

1. 3
2. 4
3. 5
4. 6
5. 7

Now what if our original function had multiple arguments? `lapply()` actually let's us deal with that by simply adding them in like this:

In [39]:

```
add_choice <- function(num,choice) {  
  return(num+choice)  
}  
  
add_choice(2,3)
```

Out[39]:

5

In [40]:

```
# Uh oh! Forgot to add other arguments!  
lapply(v,add_choice)
```

Error in FUN(X[[i]], ...): argument "choice" is missing, with no default

In [41]:

```
# Nice!  
lapply(v,add_choice,choice=10)
```

Out[41]:

1. 11
2. 12
3. 13
4. 14
5. 15

You can do this with several arguments, you just keep adding them.

## sapply() vs. lapply()

Notice that `lapply` returned a list, we can use `sapply`, which simplifies the process by returning a vector or matrix. For example:

In [45]:

```
help(sapply)
```

Out[45]:

|               |                 |
|---------------|-----------------|
| lapply {base} | R Documentation |
|---------------|-----------------|

## Apply a Function over a List or Vector

## Description

`lapply` returns a list of the same length as `X`, each element of which is the result of applying `FUN` to the corresponding element of `X`.

`sapply` is a user-friendly version and wrapper of `lapply` by default returning a vector, matrix or, if `simplify = "array"`, an array if appropriate, by applying `simplify2array()`. `sapply(x, f, simplify = FALSE, USE.NAMES = FALSE)` is the same as `lapply(x, f)`.

`vapply` is similar to `sapply`, but has a pre-specified type of return value, so it can be safer (and sometimes faster) to use.

`replicate` is a wrapper for the common use of `sapply` for repeated evaluation of an expression (which will usually involve random number generation).

`simplify2array()` is the utility called from `sapply()` when `simplify` is not false and is similarly called from `mapply()`.

## Usage

```
lapply(X, FUN, ...)
```

```
sapply(X, FUN, ..., simplify = TRUE, USE.NAMES = TRUE)
```

```
vapply(X, FUN, FUN.VALUE, ..., USE.NAMES = TRUE)
```

```
replicate(n, expr, simplify = "array")
```

```
simplify2array(x, higher = TRUE)
```

## Arguments

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>X</code>         | a vector (atomic or list) or an expression object. Other objects (including classed objects) will be coerced by <code>base::as.list</code> .                                                                                                                                                                                                                                                                                                        |
| <code>FUN</code>       | the function to be applied to each element of <code>X</code> : see ‘Details’. In the case of functions like <code>+</code> , <code>%*%</code> , the function name must be backquoted or quoted.                                                                                                                                                                                                                                                     |
| <code>...</code>       | optional arguments to <code>FUN</code> .                                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>simplify</code>  | logical or character string; should the result be simplified to a vector, matrix or higher dimensional array if possible? For <code>sapply</code> it must be named and not abbreviated. The default value, <code>TRUE</code> , returns a vector or matrix if appropriate, whereas if <code>simplify = "array"</code> the result may be an array of “rank” ( <code>=length(dim(.))</code> ) one higher than the result of <code>FUN(X[[i]])</code> . |
| <code>USE.NAMES</code> | logical; if <code>TRUE</code> and if <code>X</code> is character, use <code>X</code> as names for the result unless it had names already. Since this argument follows <code>...</code> its name cannot be abbreviated.                                                                                                                                                                                                                              |
| <code>FUN.VALUE</code> | a (generalized) vector; a template for the return value from <code>FUN</code> . See ‘Details’.                                                                                                                                                                                                                                                                                                                                                      |
| <code>n</code>         | integer: the number of replications.                                                                                                                                                                                                                                                                                                                                                                                                                |
| <code>expr</code>      | the expression (a language object, usually a call) to evaluate repeatedly.                                                                                                                                                                                                                                                                                                                                                                          |
| <code>x</code>         | a list, typically returned from <code>lapply()</code> .                                                                                                                                                                                                                                                                                                                                                                                             |
| <code>higher</code>    | logical; if true, <code>simplify2array()</code> will produce a (“higher rank”) array when appropriate, whereas <code>higher = FALSE</code> would return a matrix (or vector) only. These two cases correspond to <code>sapply(*, simplify = "array")</code> or <code>simplify = TRUE</code> , respectively.                                                                                                                                         |

## Details

`FUN` is found by a call to `match.fun` and typically is specified as a function or a symbol (e.g., a backquoted name) or a character string specifying a function to be searched for from the environment of the call to `lapply`.

Function `FUN` must be able to accept as input any of the elements of `X`. If the latter is an atomic vector, `FUN` will always be passed a length-one vector of the same type as `X`.

Arguments in `...` cannot have the same name as any of the other arguments, and care may be needed to avoid partial matching to `FUN`. In general-purpose code it is good practice to name the first two arguments `X` and `FUN` if `...` is passed through: this both avoids partial matching to `FUN` and ensures that a sensible error message is given if arguments named `X` or `FUN` are passed through `...`.

Simplification in `sapply` is only attempted if `X` has length greater than zero and if the return values from all elements of `X` are all of

simplification in `sapply` is only attempted if the length greater than zero and if the return values from all elements of `X` are all of the same (positive) length. If the common length is one the result is a vector, and if greater than one is a matrix with a column corresponding to each element of `X`.

Simplification is always done in `vapply`. This function checks that all values of `FUN` are compatible with the `FUN.VALUE`, in that they must have the same length and type. (Types may be promoted to a higher type within the ordering logical < integer < double < complex, but not demoted.)

Users of S4 classes should pass a list to `lapply` and `vapply`: the internal coercion is done by the `as.list` in the base namespace and not one defined by a user (e.g., by setting S4 methods on the base function).

`lapply` and `vapply` are primitive functions.

## Value

For `lapply`, `sapply(simplify = FALSE)` and `replicate(simplify = FALSE)`, a list.

For `sapply(simplify = TRUE)` and `replicate(simplify = TRUE)`: if `X` has length zero or `n = 0`, an empty list. Otherwise an atomic vector or matrix or list of the same length as `X` (of length `n` for `replicate`). If simplification occurs, the output type is determined from the highest type of the return values in the hierarchy `NULL < raw < logical < integer < double < complex < character < list < expression`, after coercion of pairlists to lists.

`vapply` returns a vector or array of type matching the `FUN.VALUE`. If `length(FUN.VALUE) == 1` a vector of the same length as `X` is returned, otherwise an array. If `FUN.VALUE` is not an array, the result is a matrix with `length(FUN.VALUE)` rows and `length(X)` columns, otherwise an array `a` with `dim(a) == c(dim(FUN.VALUE), length(X))`.

The (Dim)names of the array value are taken from the `FUN.VALUE` if it is named, otherwise from the result of the first function call. Column names of the matrix or more generally the names of the last dimension of the array value or names of the vector value are set from `X` as in `sapply`.

## Note

`sapply(*, simplify = FALSE, USE.NAMES = FALSE)` is equivalent to `lapply(*)`.

For historical reasons, the calls created by `lapply` are unevaluated, and code has been written (e.g., `bquote`) that relies on this. This means that the recorded call is always of the form `FUN(X[[i]], ...)`, with `i` replaced by the current (integer or double) index. This is not normally a problem, but it can be if `FUN` uses `sys.call` or `match.call` or if it is a primitive function that makes use of the call. This means that it is often safer to call primitive functions with a wrapper, so that e.g. `lapply(1:1, function(x) is.numeric(x))` is required to ensure that method dispatch for `is.numeric` occurs correctly.

If `expr` is a function call, be aware of assumptions about where it is evaluated, and in particular what `...` might refer to. You can pass additional named arguments to a function call as additional named arguments to `replicate`: see 'Examples'.

## References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

## See Also

`apply`, `tapply`, `mapply` for applying a function to multiple arguments, and `rapply` for a recursive version of `lapply()`, `eapply` for applying a function to each entry in an environment.

## Examples

```
require(stats); require(graphics)

x <- list(a = 1:10, beta = exp(-3:3), logic = c(TRUE,FALSE,FALSE,TRUE))
# compute the list mean for each list element
lapply(x, mean)
# median and quartiles for each list element
lapply(x, quantile, probs = 1:3/4)
sapply(x, quantile)
i39 <- sapply(3:9, seq) # list of vectors
sapply(i39, fivenum)
vapply(i39, fivenum,
      c(Min. = 0, "1st Qu." = 0, Median = 0, "3rd Qu." = 0, Max. = 0))

## sapply(*, "array") -- artificial example
#> array(1:10, dim=c(10L, 1L), dimnames=list(1:10, ""))
```

```
(v <- structure(10*(5:8), names = LETTERS[1:4]))
f2 <- function(x, y) outer(rep(x, length.out = 3), y)
(a2 <- sapply(v, f2, y = 2*(1:5), simplify = "array"))
a.2 <- vapply(v, f2, outer(1:3, 1:5), y = 2*(1:5))
stopifnot(dim(a2) == c(3,5,4), all.equal(a2, a.2),
          identical(dimnames(a2), list(NULL,NULL,LETTERS[1:4])))
```

```
hist(replicate(100, mean(rexp(10))))
```

```
## use of replicate() with parameters:
foo <- function(x = 1, y = 2) c(x, y)
# does not work: bar <- function(n, ...) replicate(n, foo(...))
bar <- function(n, x) replicate(n, foo(x = x))
bar(5, x = 3)
```

[Package *base* version 3.2.2]

In [46]:

```
# Nice! A vector returned
sapply(v, add_choice, choice=10)
```

Out[46]:

```
11 12 13 14 15
```

In [53]:

```
# let's prove it to ourselves
lapp <- lapply(v, add_choice, choice=10)
sapp <- sapply(v, add_choice, choice=10)

class(lapp) # a list
class(sapp) # vector of numerics
```

Out[53]:

```
'list'
```

Out[53]:

```
'numeric'
```

## sapply() limitations

sapply() won't be able to automatically return a vector if your applied function doesn't return something for all elements in that vector. For example:

In [60]:

```
# Checks for even numbers
even <- function(x) {
  return(x[(x %% 2 == 0)])
}

nums <- c(1,2,3,4,5)

sapply(nums, even)
```

Out[60]:



```
1.  
2. 2  
3.  
4. 4  
5.
```

In [61]:

```
lapply(nums, even)
```

Out[61]:

```
1.  
2. 2  
3.  
4. 4  
5.
```

## Other *apply()* type functions

There are actually quite a few different `apply()` type functions in R. We've gone over everything you need to know for now. But if your curious in finding out more about them you can check out this [documentation](#) or this excellent [StackOverflow](#) answer, copied here below.

R has many *\*apply* functions which are ably described in the help files (e.g. `?apply`). There are enough of them, though, that beginning useRs may have difficulty deciding which one is appropriate for their situation or even remembering them all. They may have a general sense that "I should be using an *\*apply* function here", but it can be tough to keep them all straight at first.

Despite the fact (noted in other answers) that much of the functionality of the *\*apply* family is covered by the extremely popular `plyr` package, the base functions remain useful and worth knowing.

This answer is intended to act as a sort of **signpost** for new useRs to help direct them to the correct *\*apply* function for their particular problem. Note, this is **not** intended to simply regurgitate or replace the R documentation! The hope is that this answer helps you to decide which *\*apply* function suits your situation and then it is up to you to research it further. With one exception, performance differences will not be addressed.

- **apply** - When you want to apply a function to the rows or columns of a matrix (and higher-dimensional analogues); not generally advisable for data frames as it will coerce to a matrix first.

```
# Two dimensional matrix  
M <- matrix(seq(1,16), 4, 4)  
  
# apply min to rows  
apply(M, 1, min)  
[1] 1 2 3 4  
  
# apply max to columns  
apply(M, 2, max)  
[1] 4 8 12 16  
  
# 3 dimensional array  
M <- array( seq(32), dim = c(4,4,2))  
  
# Apply sum across each M[, , ] - i.e Sum across 2nd and 3rd dimension  
apply(M, 1, sum)  
# Result is one-dimensional  
[1] 120 128 136 144  
  
# Apply sum across each M[, *, ] - i.e Sum across 3rd dimension  
apply(M, c(1,2), sum)  
# Result is two-dimensional  
      [,1] [,2] [,3] [,4]  
[1,]   18   26   34   42  
[2,]   20   28   36   44
```

```

[2,] 22 30 38 46
[3,] 24 32 40 48

```

If you want row/column means or sums for a 2D matrix, be sure to investigate the highly optimized, lightning-quick `colMeans`, `rowMeans`, `colSums`, `rowSums`.

- **lapply** - *When you want to apply a function to each element of a list in turn and get a list back.*

This is the workhorse of many of the other `*apply` functions. Peel back their code and you will often find `lapply` underneath.

```

x <- list(a = 1, b = 1:3, c = 10:100)
lapply(x, FUN = length)
$a
[1] 1
$b
[1] 3
$c
[1] 91

lapply(x, FUN = sum)
$a
[1] 1
$b
[1] 6
$c
[1] 5005

```

- **sapply** - *When you want to apply a function to each element of a list in turn, but you want a **vector** back, rather than a list.*

If you find yourself typing `unlist(lapply(...))`, stop and consider `sapply`.

```

x <- list(a = 1, b = 1:3, c = 10:100)
#Compare with above; a named vector, not a list
sapply(x, FUN = length)
a b c
1 3 91

sapply(x, FUN = sum)
a b c
1 6 5005

```

In more advanced uses of `sapply` it will attempt to coerce the result to a multi-dimensional array, if appropriate. For example, if our function returns vectors of the same length, `sapply` will use them as columns of a matrix:

```

sapply(1:5, function(x) rnorm(3,x))

```

If our function returns a 2 dimensional matrix, `sapply` will do essentially the same thing, treating each returned matrix as a single long vector:

```

sapply(1:5, function(x) matrix(x,2,2))

```

Unless we specify `simplify = "array"`, in which case it will use the individual matrices to build a multi-dimensional array:

```

sapply(1:5, function(x) matrix(x,2,2), simplify = "array")

```

Each of these behaviors is of course contingent on our function returning vectors or matrices of the same length or dimension.

- **vapply** - *When you want to use `sapply` but perhaps need to squeeze some more speed out of your code.*

For `vapply`, you basically give R an example of what sort of thing your function will return, which can save some time coercing returned values to fit in a single atomic vector.

```

x <- list(a = 1, b = 1:3, c = 10:100)
#Note that since the advantage here is mainly speed, this
# example is only for illustration. We're telling R that
# everything returned by length() should be an integer of
# length 1.

```

```
vapply(x, FUN = length, FUN.VALUE = 0L)
a b c
1 3 91
```

- **mapply** - For when you have several data structures (e.g. vectors, lists) and you want to apply a function to the 1st elements of each, and then the 2nd elements of each, etc., coercing the result to a vector/array as in *sapply*.

This is multivariate in the sense that your function must accept multiple arguments.

```
#Sums the 1st elements, the 2nd elements, etc.
mapply(sum, 1:5, 1:5, 1:5)
[1] 3 6 9 12 15
#To do rep(1,4), rep(2,3), etc.
mapply(rep, 1:4, 4:1)
[[1]]
[1] 1 1 1 1

[[2]]
[1] 2 2 2

[[3]]
[1] 3 3

[[4]]
[1] 4
```

- **Map** - A wrapper to *mapply* with *SIMPLIFY = FALSE*, so it is guaranteed to return a list.

```
Map(sum, 1:5, 1:5, 1:5)
[[1]]
[1] 3

[[2]]
[1] 6

[[3]]
[1] 9

[[4]]
[1] 12

[[5]]
[1] 15
```

- **rapply** - For when you want to apply a function to each element of a **nested list** structure, recursively.

To give you some idea of how uncommon *rapply* is, I forgot about it when first posting this answer! Obviously, I'm sure many people use it, but YMMV. *rapply* is best illustrated with a user-defined function to apply:

```
#Append ! to string, otherwise increment
myFun <- function(x){
  if (is.character(x)){
    return(paste(x, "!", sep=""))
  }
  else{
    return(x + 1)
  }
}

#A nested list structure
l <- list(a = list(a1 = "Boo", b1 = 2, c1 = "Eeek"),
         b = 3, c = "Yikes",
         d = list(a2 = 1, b2 = list(a3 = "Hey", b3 = 5)))

#Result is named vector, coerced to character
rapply(l, myFun)
```

```
#Result is a nested list like l, with values altered
rapply(l, myFun, how = "replace")
```

- **tapply** - For when you want to apply a function to **subsets** of a vector and the subsets are defined by some other vector, usually a factor.

The black sheep of the \*apply family, of sorts. The help file's use of the phrase "ragged array" can be a bit [confusing](#), but it is actually quite simple.

A vector:

```
x <- 1:20
```

A factor (of the same length!) defining groups:

```
y <- factor(rep(letters[1:5], each = 4))
```

Add up the values in `x` within each subgroup defined by `y`:

```
tapply(x, y, sum)
  a  b  c  d  e
10 26 42 58 74
```

More complex examples can be handled where the subgroups are defined by the unique combinations of a list of several factors. `tapply` is similar in spirit to the split-apply-combine functions that are common in R (`aggregate`, `by`, `ave`, `ddply`, etc.) Hence its black sheep status.

In [ ]:

---